

```
[13]: from __future__ import annotations

import json
import time
from typing import Any, Optional

print("Ready. Python JSON + time available.")

Ready. Python JSON + time available.
```

```
•[14]: #Question 2
```

```
class Transaction:
    """Minimal payment message for SimpleBlockchain."""

    def __init__(
        self,
        sender: str,
        recipient: str,
        amount: float,
        timestamp: Optional[float] = None,
        signature: Optional[str] = None,
    ) -> None:
        self.sender = sender
        self.recipient = recipient
        self.amount = amount
        self.timestamp = time.time() if timestamp is None else float(timestamp)
        self.signature = signature

    def to_dict(self) -> dict[str, Any]:
        """Return a JSON-serialisable dictionary of all five fields."""
        return {
            "sender": self.sender,
            "recipient": self.recipient,
            "amount": self.amount,
            "timestamp": self.timestamp,
            "signature": self.signature,
        }

    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> "Transaction":
        """Construct a Transaction from a dictionary."""
        return cls(
            sender=data["sender"],
            recipient=data["recipient"],
```

```
            amount=data["amount"],
            timestamp=data.get("timestamp"),
            signature=data.get("signature"),
        )

    def to_json(self) -> str:
        """Serialise to a canonical JSON string (sort_keys for later hashing)."""
        return json.dumps(self.to_dict(), sort_keys=True, separators=(",", ":"))

    @classmethod
    def from_json(cls, payload: str) -> "Transaction":
        """Deserialise from a JSON string."""
        return cls.from_dict(json.loads(payload))

    def __repr__(self) -> str:
        return (
            f"Transaction({self.sender!r} -> {self.recipient!r}, "
            f"amount={self.amount}, signature={self.signature!r})"
        )
```

```
# Create and inspect
demo_tx = Transaction(sender="Alice", recipient="Bob", amount=10.0)
print("Object:", demo_tx)
print("Fields:")
for key, value in demo_tx.to_dict().items():
    print(f" {key!r}: {value!r}")
assert set(demo_tx.to_dict().keys()) == {
    "sender", "recipient", "amount", "timestamp", "signature"
}
print("Schema keys OK (five fields).")

Object: Transaction('Alice' -> 'Bob', amount=10.0, signature=None)
Fields:
'sender': 'Alice'
'recipient': 'Bob'
'amount': 10.0
'timestamp': 1787386457.8602927
'signature': None
Schema keys OK (five fields).
```

```
•[15]: fixed = Transaction(
    sender="Alice",
    recipient="Bob",
    amount=100.0,
```

```

•[15]: fixed = Transaction(
    sender="Alice",
    recipient="Bob",
    amount=100.0,
    timestamp=1_500_000_000.0,
    signature=None,
)

as_dict = fixed.to_dict()
print("to_dict:", as_dict)

from_dict_tx = Transaction.from_dict(as_dict)
print("from_dict:", from_dict_tx)

payload = fixed.to_json()
print("to_json (canonical):", payload)

restored = Transaction.from_json(payload)
print("from_json:", restored)

# Type fidelity checks
assert restored.sender == "Alice"
assert restored.recipient == "Bob"
assert restored.amount == 100.0

```

```

assert restored.amount == 100.0
assert restored.timestamp == 1_500_000_000.0
assert restored.signature is None
print("JSON round-trip OK (all five fields preserved; None ≠ null).")

to_dict: {'sender': 'Alice', 'recipient': 'Bob', 'amount': 100.0, 'timestamp': 1500000000.0, 'signature': None}
from_dict: Transaction('Alice' -> 'Bob', amount=100.0, signature=None)
to_json (canonical): {"amount":100.0,"recipient":"Bob","sender":"Alice","signature":null,"timestamp":1500000000.0}
from_json: Transaction('Alice' -> 'Bob', amount=100.0, signature=None)
JSON round-trip OK (all five fields preserved; None ≠ null).

```

```

•[16]: #Question 3

def validate_transaction(tx: Transaction) -> bool:
    """Structural validation only (no signature or balance checks)."""
    if not isinstance(tx.sender, str) or not tx.sender.strip():
        return False
    if not isinstance(tx.recipient, str) or not tx.recipient.strip():
        return False
    # bool is a subclass of int
    if not isinstance(tx.amount, (int, float)) or isinstance(tx.amount, bool):
        return False
    if tx.amount <= 0:
        return False
    if tx.timestamp is None:
        return False
    try:
        float(tx.timestamp)
    except (TypeError, ValueError):
        return False
    return True

```

```

ok = Transaction("Alice", "Bob", 10.0, timestamp=1.0)
print("Valid Alice+Bob:", validate_transaction(ok))

Valid Alice+Bob: True

```

```

[17]: cases = [
    ("Alice+Bob amount=10 (valid)", Transaction("Alice", "Bob", 10.0, timestamp=1.0), True),
    ("Carol amount=-5 (negative)", Transaction("Carol", "Bob", -5.0, timestamp=1.0), False),
    ("Zero amount", Transaction("Carol", "Bob", 0.0, timestamp=1.0), False),
    ("Empty sender", Transaction("", "Bob", 1.0, timestamp=1.0), False),
    ("Empty recipient", Transaction("Alice", "", 1.0, timestamp=1.0), False),
    ("Whitespace-only recipient", Transaction("Alice", " ", 1.0, timestamp=1.0), False),
]

print(f"{'Case':<36} {'got':<6} {'expected':<8} {'OK?':<4}")
print("-" * 60)
all_match = True
for label, tx, expected in cases:
    got = validate_transaction(tx)
    match = got == expected
    all_match = all_match and match
    print(f"{'label':<36} {'str(got)':<6} {'str(expected)':<8} {'match':<4}")

```

```

assert all_match
print("\nAll structural failure cases behave as expected.")

```

Case	got	expected	OK?
Alice+Bob amount=10 (valid)	True	True	True
Carol amount=-5 (negative)	False	False	True
Zero amount	False	False	True
Empty sender	False	False	True
Empty recipient	False	False	True
Whitespace-only recipient	False	False	True

All structural failure cases behave as expected.

```
[43]: class Mempool:
    """Local waiting room for structurally valid transactions."""

    def __init__(self) -> None:
        self._txs: list[Transaction] = []

    def add(self, tx: Transaction) -> bool:
        """Validate and, if valid, append. Return True iff accepted."""
        if not validate_transaction(tx):
            return False
        self._txs.append(tx)
        return True

    def get_all(self) -> list[Transaction]:
        """Return a shallow copy of pending transactions."""
        return list(self._txs)

    def __len__(self) -> int:
        return len(self._txs)

    def clear(self) -> None:
        self._txs.clear()

pool = Mempool()
```

```
print("Empty mempool size:", len(pool))
print("get_all() on empty:", pool.get_all())
```

```
Empty mempool size: 0
get_all() on empty: []
```

•[24]: #Question 4

```
# 1) Alice pays Bob
alice_to_bob = Transaction(sender="Alice", recipient="Bob", amount=10.0)
print("=== Alice -> Bob ===")
print(alice_to_bob.to_json())
accepted_alice = pool.add(alice_to_bob)
print(f"accepted: {accepted_alice}, size: {len(pool)}")
assert accepted_alice is True
assert len(pool) == 1

# 2) Carol invalid transaction
carol = Transaction(sender="Carol", recipient="Bob", amount=-5.0)
print("\n=== Carol invalid (amount=-5) ===")
accepted_carol = pool.add(carol)
print(f"accepted: {accepted_carol}, size: {len(pool)}")
assert accepted_carol is False

assert accepted_carol is False
assert len(pool) == 1 # unchanged

print("\n=== Mempool dump ===")
for tx in pool.get_all():
    print(tx.to_dict())

assert pool.get_all()[0].sender == "Alice"
print("\nScenario complete: only Alice+Bob sits in the mempool.")

=== Alice -> Bob ===
{"amount":10.0,"recipient":"Bob","sender":"Alice","signature":null,"timestamp":1787387730.9752362}
accepted: True, size: 1

=== Carol invalid (amount=-5) ===
accepted: False, size: 1

=== Mempool dump ===
{'sender': 'Alice', 'recipient': 'Bob', 'amount': 10.0, 'timestamp': 1787387730.9752362, 'signature': None}

Scenario complete: only Alice+Bob sits in the mempool.
```